In [17]:
```
import pandas as pd
import numpy as np

data = {
    "Order_ID": range(1001, 1021),
    "Order_Date": pd.date_range("2024-01-01", periods=20, freq="D"),
    "Region": ["North","South","East","West"] * 5,
    "Category": ["Electronics","Furniture","Office Supplies","Electronics","Furn
    "Sales": [45000, 23000, 12000, 52000, 28000,
              47000, 25000, 15000, 60000, 30000,
              49000, np.nan, 16000, 58000, 27000,
              51000, 26000, 14000, np.nan, 29000],
    "Quantity": [1,2,3,1,2,
                 1,2,3,1,2,
                 1,2,3,1,2,
                 1,2,3,1,2]
}

df = pd.DataFrame(data)
df
```

Out[17]:

	Order_ID	Order_Date	Region	Category	Sales	Quantity
0	1001	2024-01-01	North	Electronics	45000.0	1
1	1002	2024-01-02	South	Furniture	23000.0	2
2	1003	2024-01-03	East	Office Supplies	12000.0	3
3	1004	2024-01-04	West	Electronics	52000.0	1
4	1005	2024-01-05	North	Furniture	28000.0	2
5	1006	2024-01-06	South	Electronics	47000.0	1
6	1007	2024-01-07	East	Furniture	25000.0	2
7	1008	2024-01-08	West	Office Supplies	15000.0	3
8	1009	2024-01-09	North	Electronics	60000.0	1
9	1010	2024-01-10	South	Furniture	30000.0	2
10	1011	2024-01-11	East	Electronics	49000.0	1
11	1012	2024-01-12	West	Furniture	NaN	2
12	1013	2024-01-13	North	Office Supplies	16000.0	3
13	1014	2024-01-14	South	Electronics	58000.0	1
14	1015	2024-01-15	East	Furniture	27000.0	2
15	1016	2024-01-16	West	Electronics	51000.0	1
16	1017	2024-01-17	North	Furniture	26000.0	2
17	1018	2024-01-18	South	Office Supplies	14000.0	3
18	1019	2024-01-19	East	Electronics	NaN	1
19	1020	2024-01-20	West	Furniture	29000.0	2

In [18]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Order_ID    20 non-null     int64
 1   Order_Date  20 non-null     datetime64[ns]
 2   Region      20 non-null     object
 3   Category    20 non-null     object
 4   Sales       18 non-null     float64
 5   Quantity    20 non-null     int64
dtypes: datetime64[ns](1), float64(1), int64(2), object(2)
memory usage: 1.1+ KB
```

In [19]:
```
df.head()
```

Out[19]:

	Order_ID	Order_Date	Region	Category	Sales	Quantity
0	1001	2024-01-01	North	Electronics	45000.0	1
1	1002	2024-01-02	South	Furniture	23000.0	2
2	1003	2024-01-03	East	Office Supplies	12000.0	3
3	1004	2024-01-04	West	Electronics	52000.0	1
4	1005	2024-01-05	North	Furniture	28000.0	2

In [20]:
```
df.describe()
```

Out[20]:

	Order_ID	Order_Date	Sales	Quantity
count	20.00000	20	18.000000	20.000000
mean	1010.50000	2024-01-10 12:00:00	33722.222222	1.800000
min	1001.00000	2024-01-01 00:00:00	12000.000000	1.000000
25%	1005.75000	2024-01-05 18:00:00	23500.000000	1.000000
50%	1010.50000	2024-01-10 12:00:00	28500.000000	2.000000
75%	1015.25000	2024-01-15 06:00:00	48500.000000	2.000000
max	1020.00000	2024-01-20 00:00:00	60000.000000	3.000000
std	5.91608	NaN	15977.210077	0.767772

In [21]:
```
#checking there is any null values in the columns
df.isna().sum()
```

Out[21]:
```
Order_ID      0
Order_Date    0
Region        0
Category      0
Sales         2
Quantity      0
dtype: int64
```

In [22]:
```
#filling the null values present in sales with median of the column
df["Sales"].fillna(df["Sales"].median(),inplace=True)
```

```
C:\Users\palatherdam saikumar\AppData\Local\Temp\ipykernel_21068\1072384550.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series thr
ough chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work becau
se the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.meth
od({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df["Sales"].fillna(df["Sales"].median(),inplace=True)
```

In [23]:
```
#covert the column to datetime format

df["Order_Date"]=pd.to_datetime(df["Order_Date"])
```

In [24]:
```
#convert region and category columns to category type as there are object type b
df[["Region","Category"]]=df[["Region","Category"]].astype("category")
```

In [25]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Order_ID    20 non-null     int64
 1   Order_Date  20 non-null     datetime64[ns]
 2   Region      20 non-null     category
 3   Category    20 non-null     category
 4   Sales       20 non-null     float64
 5   Quantity    20 non-null     int64
dtypes: category(2), datetime64[ns](1), float64(1), int64(2)
memory usage: 1.1 KB
```

In [26]:
```
df["Sales"].mean()
```

Out[26]:
```
np.float64(33200.0)
```

In [27]:
```
df["Sales"].median()
```

Out[27]:
```
28500.0
```

In [28]:
```
#if mean>median then it is right skewed that is most of the data lies on the lef
```

In [29]:
```
df["Sales"].std()
```

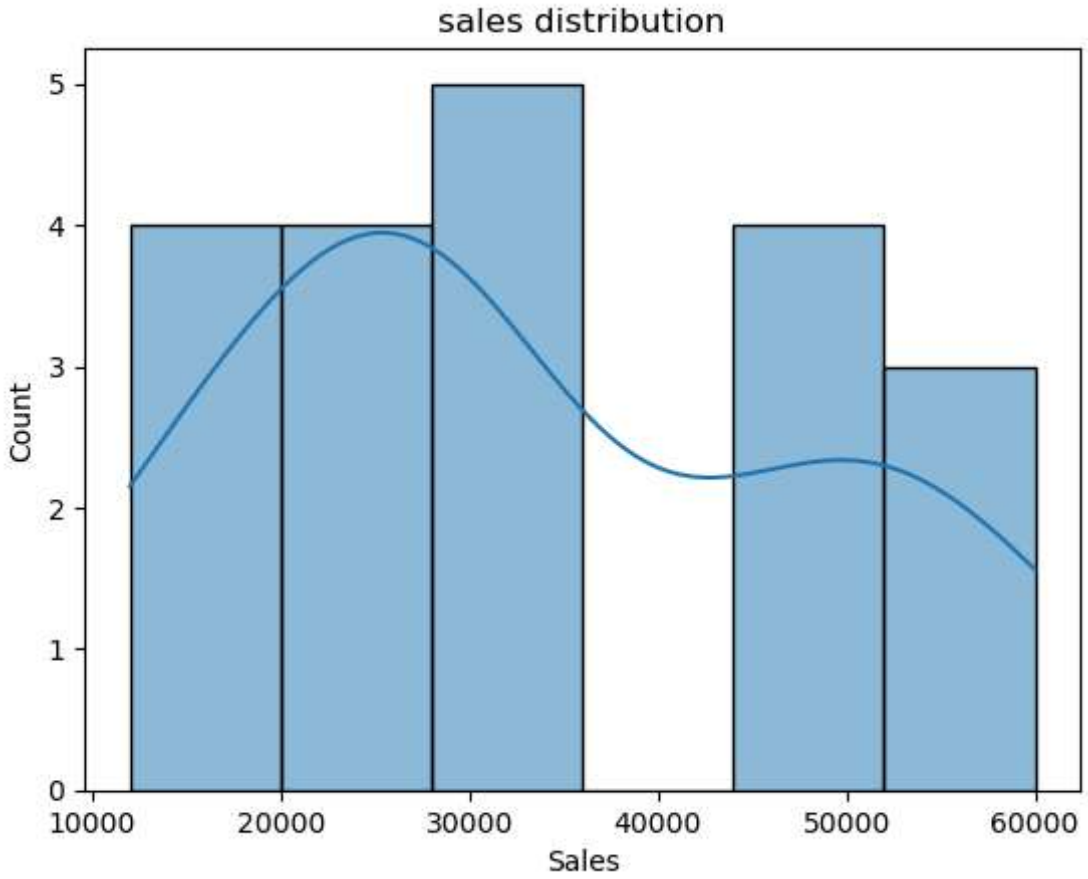
Out[29]:
```
15198.164709145502
```

In [30]:
```
df["Sales"].quantile([0.25, 0.5, 0.75])
```

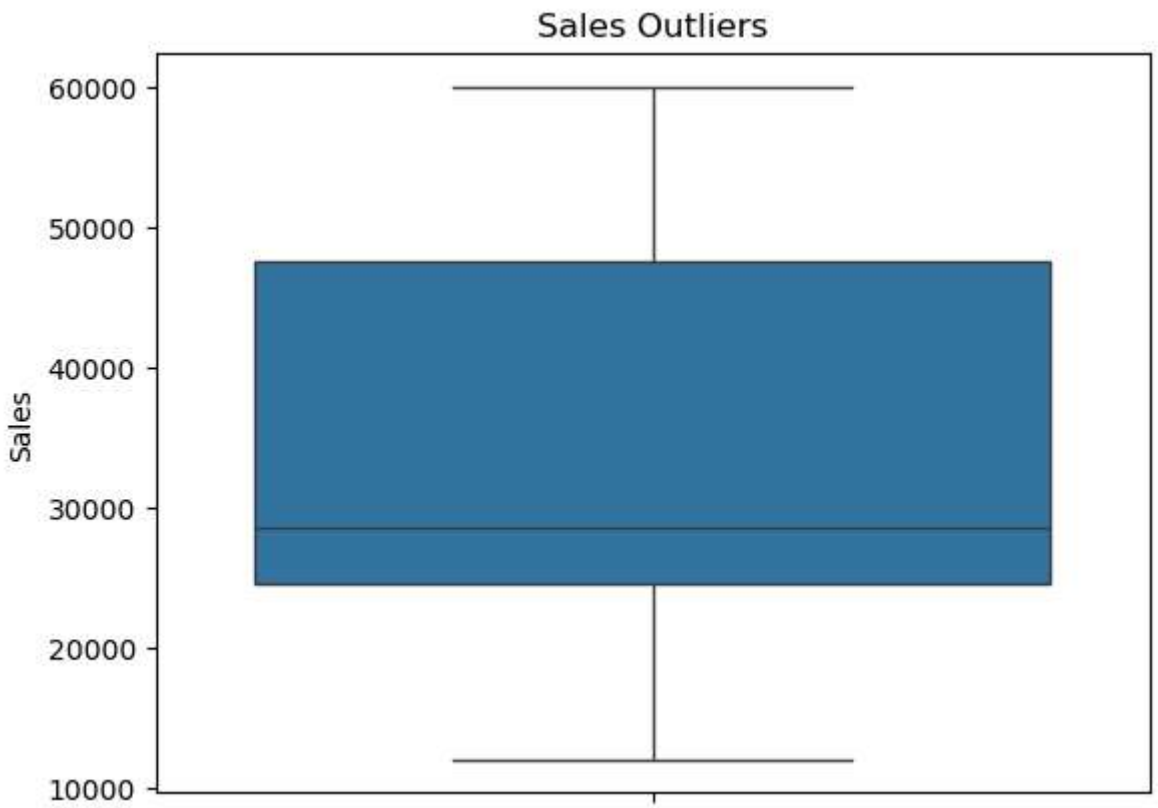
Out[30]:
```
0.25    24500.0
0.50    28500.0
0.75    47500.0
Name: Sales, dtype: float64
```

In [33]:
```
#this shows that there are outliers and that spereading of the data
#visualize it using matplotlib and seaborn now
```

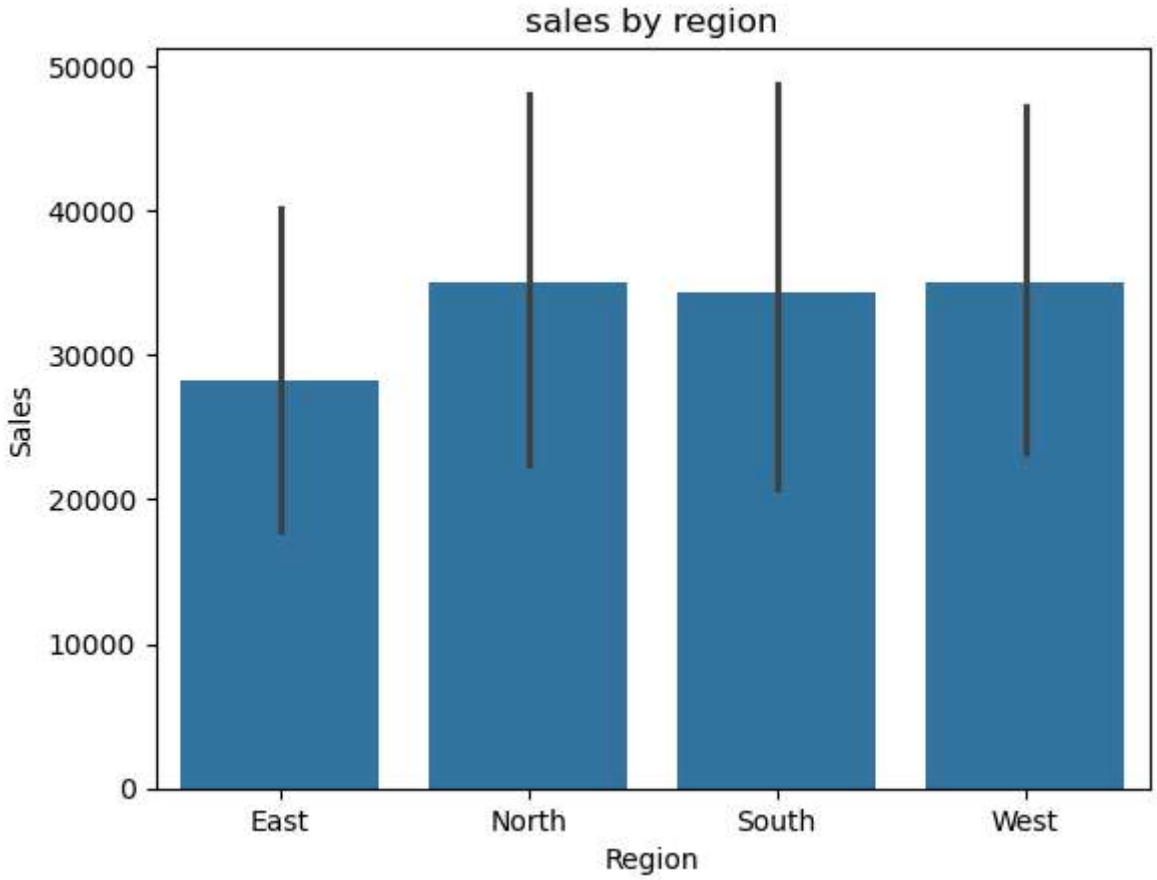
In [37]:
```
import seaborn as sns
import matplotlib.pyplot as plt
sns.histplot(df["Sales"],kde=True)
plt.title("sales distribution")
plt.show()
```



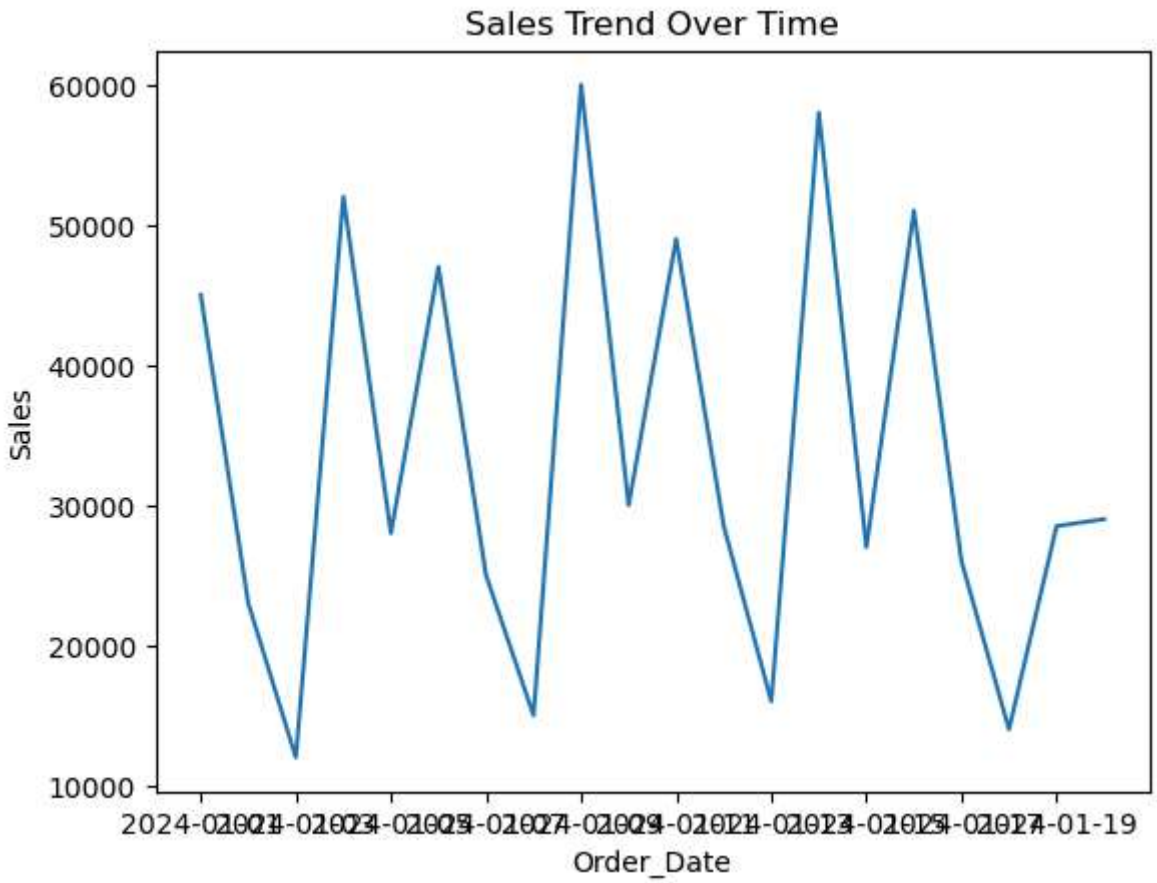
In [38]:
```
sns.boxplot(y=df["Sales"])
plt.title("Sales Outliers")
plt.show()
```



In [39]:
```
sns.barplot(data=df,x="Region",y="Sales")
plt.title("sales by region")
plt.show()
```



In [40]:
```
sns.lineplot(data=df, x="Order_Date", y="Sales")
plt.title("Sales Trend Over Time")
plt.show()
```



In [ ]:
```
#insights from the above eda tells “The West and North regions exhibit nearly eq
#The sales data is right-skewed, with most values clustered at lower ranges and
#The box plot does not reveal any extreme outliers; however, sales consistency s
#standard deviation rather than the absence of outliers alone.”
```

In [42]:
```
#this is aggregation process that tells which category is dominated in each regi
region_category_sales = (
    df.groupby(["Region", "Category"])["Sales"]
      .mean()
      .reset_index()
)

region_category_sales
```

```
C:\Users\palatherdam saikumar\AppData\Local\Temp\ipykernel_21068\2807916086.py:3:
FutureWarning: The default of observed=False is deprecated and will be changed to
True in a future version of pandas. Pass observed=False to retain current behavio
r or observed=True to adopt the future default and silence this warning.
  df.groupby(["Region", "Category"])["Sales"]
```

Out[42]:

	Region	Category	Sales
0	East	Electronics	38750.0
1	East	Furniture	26000.0
2	East	Office Supplies	12000.0
3	North	Electronics	52500.0
4	North	Furniture	27000.0
5	North	Office Supplies	16000.0
6	South	Electronics	52500.0
7	South	Furniture	26500.0
8	South	Office Supplies	14000.0
9	West	Electronics	51500.0
10	West	Furniture	28750.0
11	West	Office Supplies	15000.0

In [43]:
```
pivot_table=region_category_sales.pivot(
    index="Region",
    columns="Category",
    values="Sales"
)
pivot_table
```

Out[43]:

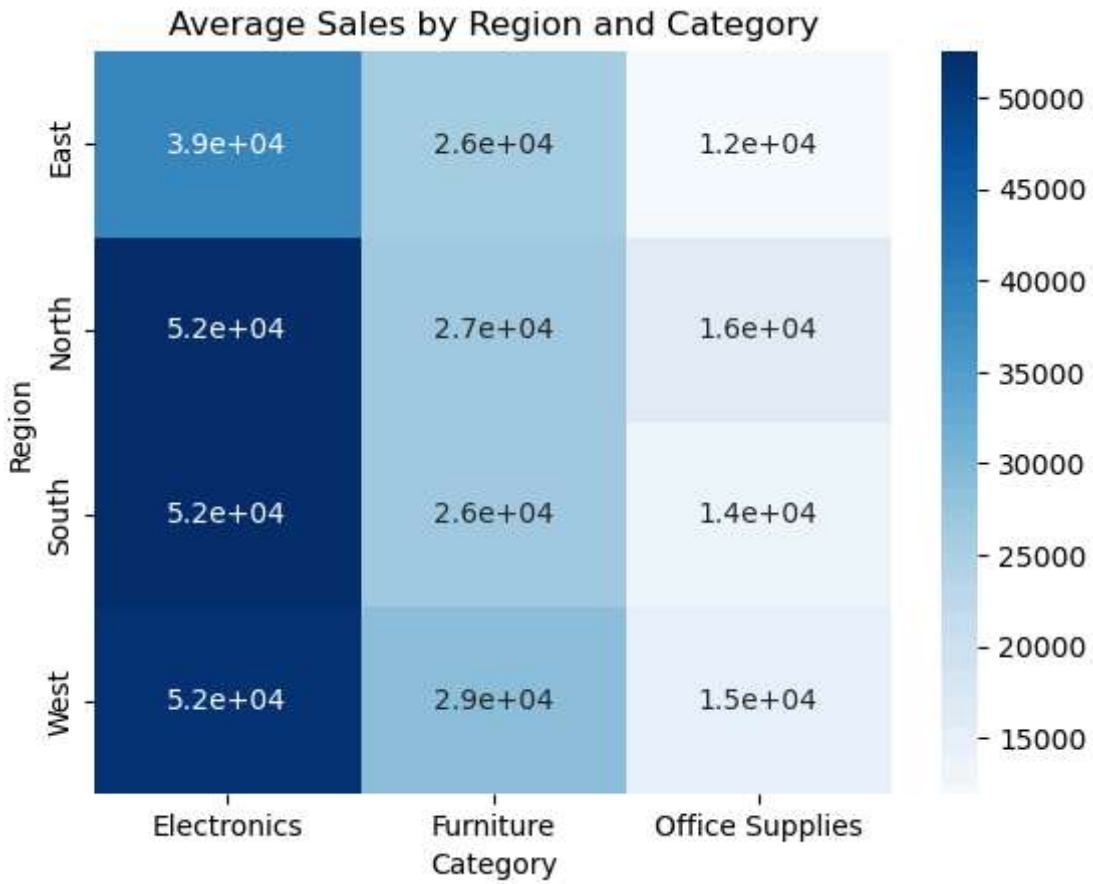
Category	Electronics	Furniture	Office Supplies
Region			
East	38750.0	26000.0	12000.0
North	52500.0	27000.0	16000.0
South	52500.0	26500.0	14000.0
West	51500.0	28750.0	15000.0

In [44]:

```
import seaborn as sns
import matplotlib.pyplot as plt

pivot_rc = region_category_sales.pivot(
    index="Region",
    columns="Category",
    values="Sales"
)

sns.heatmap(pivot_rc, annot=True, cmap="Blues")
plt.title("Average Sales by Region and Category")
plt.show()
```



In [45]:

```
#now corelation analysis
#this corelation answers is quantity related to sales,if yes then how
```

In [46]:

```
df["Sales"].corr(df["Quantity"])
```

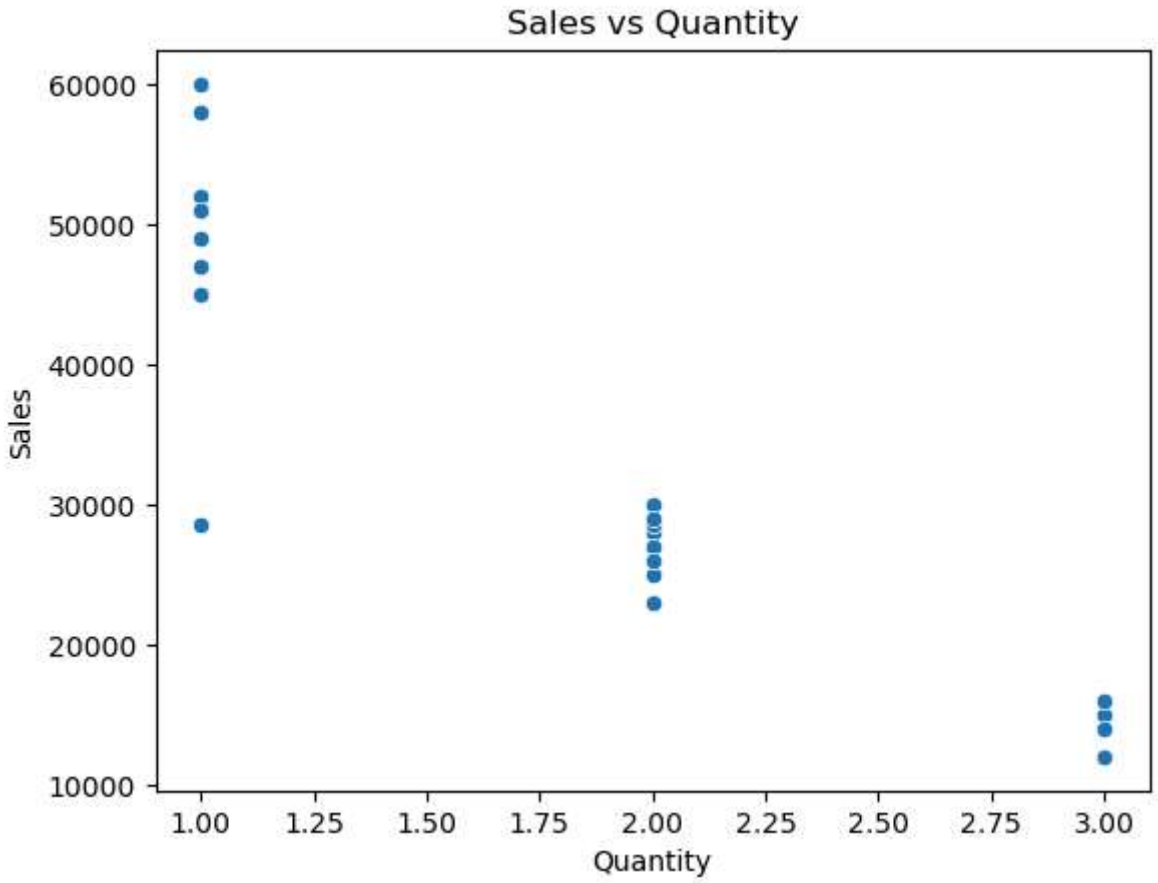
Out[46]:

```
np.float64(-0.9052539194350825)
```

In [47]:
```
#this shows negitive relationship bcoz the value is -ve if it is 0 then no relat
```

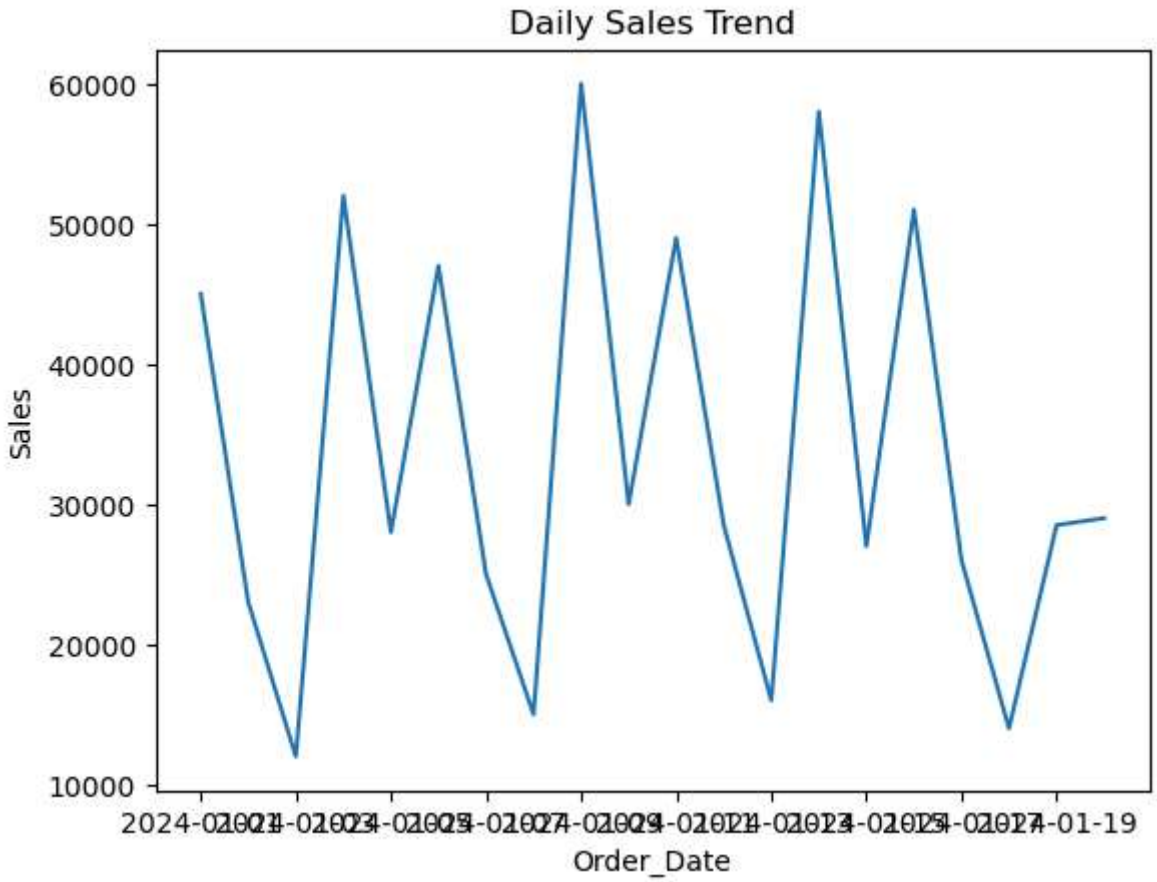
In [48]:
```
#lets conform it via visuaization
```

In [49]:
```
sns.scatterplot(data=df, x="Quantity", y="Sales")
plt.title("Sales vs Quantity")
plt.show()
```



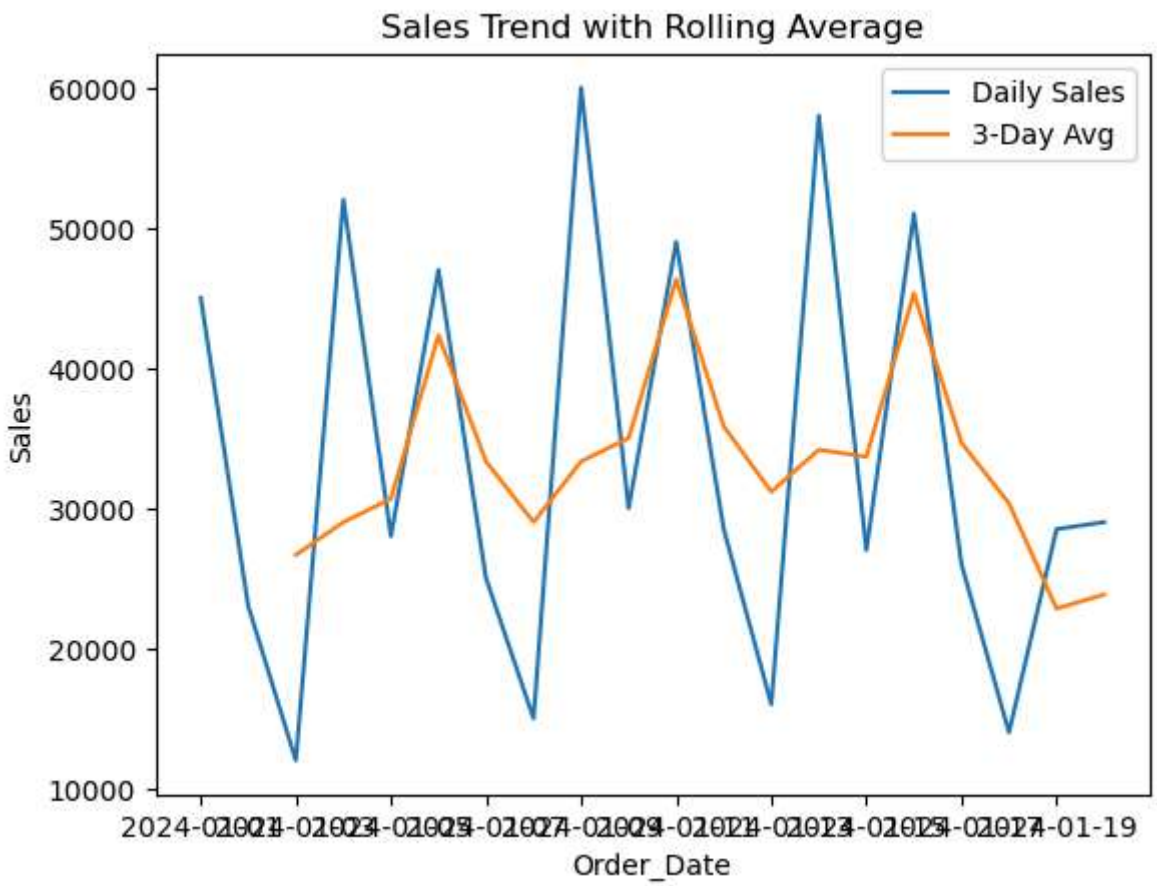
In [50]:
```
#now lets check is perfomance stable over time
```

In [51]:
```
sns.lineplot(data=df, x="Order_Date", y="Sales")
plt.title("Daily Sales Trend")
plt.show()
```



In [52]:
```
df_sorted = df.sort_values("Order_Date")
df_sorted["Rolling_3Day_Avg"] = df_sorted["Sales"].rolling(3).mean()

sns.lineplot(x=df_sorted["Order_Date"], y=df_sorted["Sales"], label="Daily Sales
sns.lineplot(x=df_sorted["Order_Date"], y=df_sorted["Rolling_3Day_Avg"], label="
plt.legend()
plt.title("Sales Trend with Rolling Average")
plt.show()
```



In [53]:
```
#Rolling line(orange) smoother → stable trend
#Sharp spikes → volatility
```

In [ ]:
```
#Regional Performance

#West and North regions show comparable average sales, indicating similar market

#Category Performance

#Electronics consistently generate higher sales across most regions, suggesting

#Distribution Insight

#Sales data is right-skewed, implying that a few high-value orders influence the

#Relationship Insight

#Sales and quantity show weak correlation, suggesting revenue is driven more by

#Trend Insight

#Sales trend appears relatively stable over time with minor short-term fluctuati
```